

SQL Terms:

- **What is Data?**
- Data is a raw form of knowledge. Data can come in the form of text, observations, figures, images, numbers, graphs, or symbols. For example, data might include individual prices, weights, addresses, ages, names, temperatures, dates, or distances. When we process data we get information.
- **What is a database?**
- Ans: SQL database is a structured approach to storing data and performing operations using structured query language for data creation, modification & its organization. Using this approach the data can be easily controllable/suitable format.
- **There are 5 major types of databases:**
- **Hierarchical:** The data are organized into a tree-like structure. It is developed by IBM in 1960. Ex: The IBM Information Management System (IMS) and RDM Mobile
- **Network:** A database organized according to ownership of records, Integrated Data Store (IDS), Integrated Data Store II (IDS II)
- **Object Oriented:** information is represented in the form of objects, Ex: ObjectDB, db4o
- **Relational:** A relational database is a type of database that stores and provides access to data points that are related to one another. Data is represented in tabular format.
- Each row in the table is a record with a unique ID called the key. The columns of the table hold attributes/ fields of the data
- **NoSQL:** NoSQL databases (aka "not only SQL") are non-tabular databases and store data differently than relational tables. They store data in the form of document or key-value pair. Ex: MongoDB, Redis.
- **What is DBMS?**
- Database Management Systems (DBMS) are software systems used to store, retrieve, and run queries on data. A DBMS serves as an interface between an end-user and a database, allowing users to create, read, update, and delete data in the database.

SQL Terms:

- Advantages of Relational database:
 1. Memory Management
 2. Data readiability
 3. Scalabilty : handles large amount of data
 4. Data indexing: faster retrieval of data
 5. Data security
- RDBMS examples = MySQL,MS SQL server, Oracle,PostgreSQL, MariaDB
- MySQL: It is an open source (code that is designed to be publicly accessible—anyone can use), cross-platform RDBMS developed by Oracle corporation that is based on SQL.
- Diffrence between SQL & MySQL : SQL is a query programming language that manages RDBMS where MySQL is a relational database management system that uses SQL.
- Query:It is a command or inquiry about set of data.
- Database & Schema: A database is the main container, it contains the data and log files, and all the schemas within it.
- Schemas are like folders within a database, and are mainly used to group logical objects together.

Queries:

- SQL commands are like instructions to a table. It is used to interact with the database perform various tasks like creating a table, adding data to tables, dropping the table, modifying the table, set permission for users.
- SQL is not a case sensitive language.

Data types in Sql:

- String Data types

Char(size), varchar(size), binary(size) ,varbinary(size), blob, text.

- Numeric Data types

Int(size), bit(size)

- Date and time Data types

date, datetime, time, year

- it will always take 10 bytes of data regardless of whether you store one character, five characters, seven characters, or ten characters. It will always take 10 bytes, i.e., you can store a maximum of 10 characters in a column. Due to this, sometimes, a lot of database space is wasted.
- Suppose you declare a variable of varchar (10) data type; then it will use the number of bytes equal to the number of characters. So, storing two characters will take two bytes; if you are storing five characters, it will take 5 bytes, and so on. Due to this, much database space is saved, which was wasted when using the Char data type.

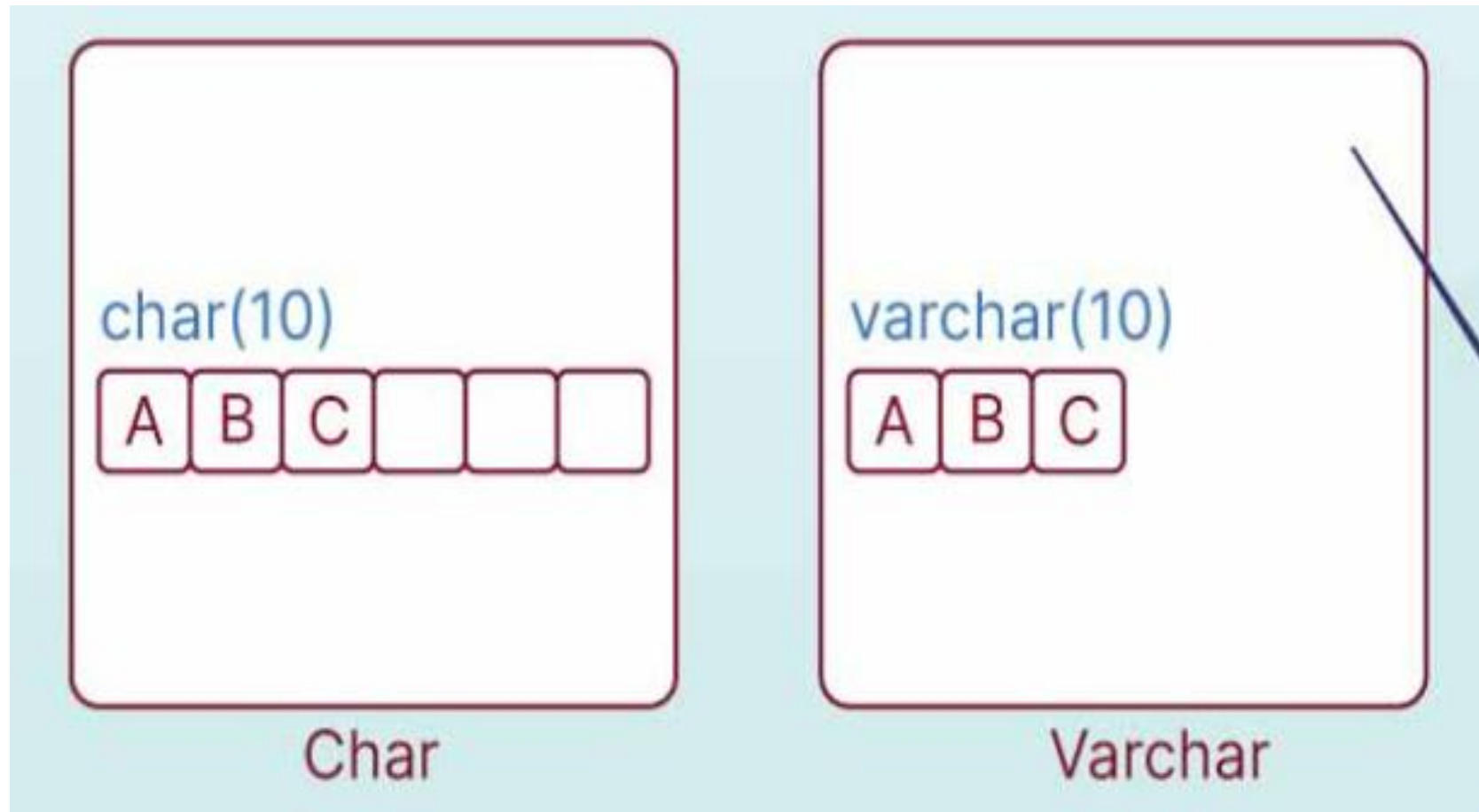
String Data Type:

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The <i>size</i> parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The <i>size</i> parameter specifies the maximum column length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The <i>size</i> parameter specifies the column length in bytes. Default is 1
VARBINARY(size)	Equal to VARCHAR(), but stores binary byte strings. The <i>size</i> parameter specifies the maximum column length in bytes.
TINYBLOB	For BLOBs (Binary Large Objects). Max length: 255 bytes
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT(size)	Holds a string with a maximum length of 65,535 bytes
<u>BLOB(size)</u>	<u>For BLOBs (Binary Large Objects). Holds up to 65,535 bytes of data</u>

Difference between char() & varchar():

Benchmark	Char	Varchar
Stands for	Char stands for “Character.”	Varchar stands for “Variable Character.”
Use	Stores characters or strings of a fixed length (size).	Stores characters or strings of a variable length (size).
Storage size	Is equal to n bytes, i.e., set length.	Is equal to the actual length of the entered string in bytes.
Performance	Provides better performance in comparison to Varchar.	Provides inferior performance as length has to be accounted for.
Padding	Pad space is required when storing strings less than the fixed size length.	No padding is required as it is variable in size.
Byte	Takes 1 byte for each character.	Takes 1 byte for each character, and in addition to that, it also takes some extra bytes for holding length information.
Best to use it	When we expect the data values in a column to be of the same length	When we expect the data values in a column to be of variable length.

Difference between char() & varchar():



Numeric Data types:

BIT(<i>size</i>)	A bit-value type. The number of bits per value is specified in <i>size</i> . The <i>size</i> parameter can hold a value from 1 to 64. The default value for <i>size</i> is 1.
TINYINT(<i>size</i>)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The <i>size</i> parameter specifies the maximum display width (which is 255)
BOOL	Zero is considered as false, nonzero values are considered as true.
BOOLEAN	Equal to BOOL
SMALLINT(<i>size</i>)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The <i>size</i> parameter specifies the maximum display width (which is 255)
MEDIUMINT(<i>size</i>)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The <i>size</i> parameter specifies the maximum display width (which is 255)
INT(<i>size</i>)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The <i>size</i>

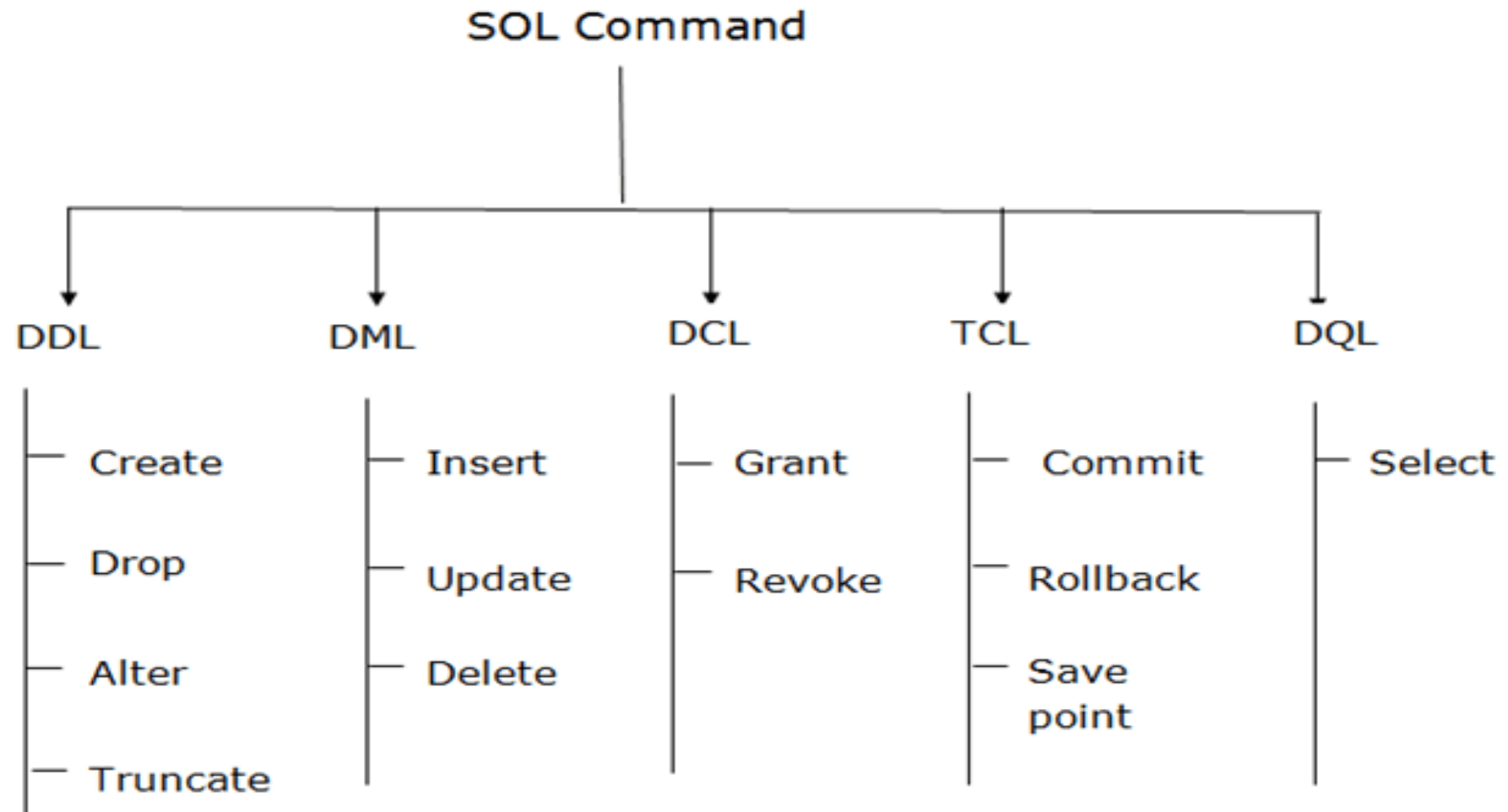
Date and time Data types

DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(<i>fsp</i>)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(<i>fsp</i>)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(<i>fsp</i>)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000.

SQL Commands:

1. DDL – Data Definition Language
2. DQL – Data Query Language
3. DML – Data Manipulation Language
4. DCL – Data Control Language
5. TCL – Transaction Control Language

SQL Commands:



DQL:

- It is used to fetch data from table / db based on our requirement.
- It contains only one query named “select”.
- Select command is used to select / retrieve attributes/ attribute based on some condition.
- Select * from user;
- Select name from user;
- Select name, id from user;

Where Clause:

- The WHERE clause is used to filter records.
- It is used to extract only those records that fulfill a specified condition.
- for retrieving data according to some conditions.

Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Comparison Operators:

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

Logical Operators:

- The WHERE clause can be combined with AND, OR, and NOT operators.
- The AND and OR operators are used to filter records based on more than one condition:
- The AND operator displays a record if all the conditions separated by AND are TRUE.
- The OR operator displays a record if any of the conditions separated by OR is TRUE.
- The NOT operator displays a record if the condition(s) is NOT TRUE.

AND Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 AND condition2 AND condition3 ...;
```

OR Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 OR condition2 OR condition3 ...;
```

NOT Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE NOT condition;
```


BETWEEN:

- BETWEEN in SQL is a logical operator that is used to select a range of values from the database table.

BETWEEN Syntax

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name BETWEEN value1 AND value2;
```

- These values are used to define the range for validating the expression. The range is inclusive.
- select * from user where id between id1 AND id2;
- select * from user where id NOT between id1 AND id2;

IN Operator:

- The IN operator allows you to specify multiple values in a WHERE clause.
- The IN operator is a shorthand for multiple OR conditions, means It provides an easy way to handle multiple OR conditions.

Syntax

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name IN (value1, value2, ...);
```

- select * from student where id in(id1,id2,id3,id4);
- select * from student where id not in(id1,id2,id3);
- select * from student where address not in(value1, value2, value3);

Like Operator:

- The **SQL LIKE** operator is used to retrieve the data in a column of a table, based on a specified pattern.
- It is used along with the WHERE clause of the UPDATE, DELETE and SELECT statements, to filter the rows based on the given pattern. These patterns are specified using **Wildcards**.
- Wildcards are special characters that can stand in for unknown characters in a text value .

Syntax

The basic syntax of the SQL LIKE operator is as follows –

```
SELECT column1, column2, ...  
FROM table_name  
WHERE columnn LIKE specified_pattern;
```

- the % is a wild card which represents zero, one or multiple characters. Ex: the expression K% specifies that it will display the list of all the students whose name starts with 'K'.

DELETE

26-04-24

DELETE Syntax

```
DELETE FROM table_name WHERE condition;
```

ALTER

- The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.

```
ALTER TABLE table_name  
ADD column_name datatype;
```

```
ALTER TABLE table_name  
MODIFY COLUMN column_name datatype;
```

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Alter:

- ALTER TABLE `demo`.`demo\_details` CHANGE COLUMN `newage` `age1` INT(11) DEFAULT NULL;

Update:

- MySQL UPDATE query is a DML statement used to modify the data of the MySQL table within the database.
- The UPDATE statement is used with the **SET** and **WHERE** clauses. The SET clause is used to change the values of the specified column. We can update single or multiple columns at a time.

UPDATE Syntax

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

RENAME TABLE & COLUMN

- ALTER TABLE table_name CHANGE old_column_name new_column_name datatype ;
- ALTER TABLE old_name RENAME new_name;

Constraints

27-04-24

- Used to specify rules for the data in a table
- It ensures the accuracy of the records in a table.
- If there is any violation between constraint and the record then the action is aborted.

Not null

- Ensures that a column cannot have a NULL value.
- CREATE TABLE Student (ID int NOT NULL, LastName varchar(255) NOT NULL, FirstName varchar(255), Age int);

UNIQUE

- - Ensures that all values in a column are different.
- Null values are allowed.
- CREATE TABLE Student (ID int NOT NULL, LastName varchar(255) NOT NULL, Age int, UNIQUE (ID));
- OR by another way:
- CREATE TABLE Student (ID int NOT NULL UNIQUE, LastName varchar(255) NOT NULL, Age int);

Primary Key

- The primary key constraint uniquely identifies each record in a table.
- Primary keys must contain UNIQUE values, and cannot contain NULL values.
- A table can only have one primary key, but it can have multiple unique constraints.

Difference between unique & Primary key:

unique

- Null values can be inserted.
- A table can have multiple unique Constraints.

Primary key

- Null values not allowed.
- A table can have only one primary key.

AUTO_INCREMENT

29-04-24

- The auto increment in SQL is a constraint that is applied to a field so that it can automatically generate and provide a unique value to every record that you enter into an SQL table.
- This field is often used as the **PRIMARY KEY** column, where you need to provide a unique value for every record you add.
- **AUTO_INCREMENT** is the keyword used for this feature.
- Starting value=1 by default & it will increment by 1 for each new record.

AUTO_INCREMENT

- When you insert any other value into an `AUTO_INCREMENT` column, the column is set to that value and the sequence is reset so that the next automatically generated value.

FOREIGN KEY Constraint

- What is relational databases?
- It is the field in one table that refers to the Primary key in another table.
- The table with the foreign key → Child table.
- The table with the primary key → referenced or Parent table.

Difference between Primary key & Foreign key:

1. A primary key is a unique identifier for each record in a table.
2. Does not allow NULL values in the primary key field.
3. Only one primary key can exist per table
4. No duplicates are allowed.

1. A foreign key establishes a relationship between tables by referencing the primary key of another table.
2. Allows NULL values in the foreign key field, indicating a missing values.
3. Multiple foreign keys can exist within a table, depending on the relationships with other tables.
4. Can contain duplicate foreign key values, reflecting multiple records associated with the same reference.

Revision:
30-04-24

- create table customer(cid int primary key, cname varchar(30));
- create table product(pid int primary key, pname varchar(30), price int, cid int, FOREIGN KEY(CID) references customer(cid));

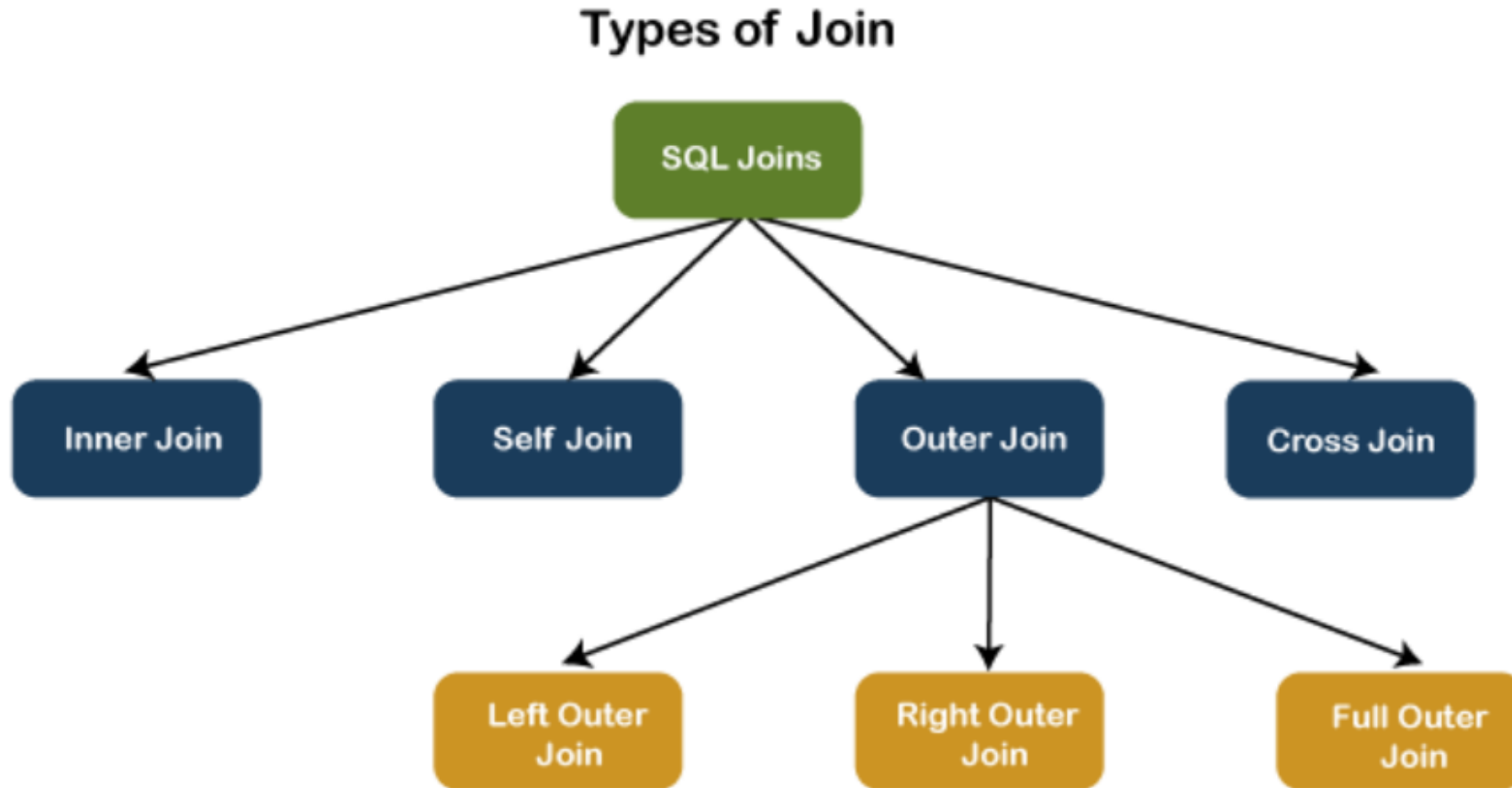
-----Ex:2-----

- create table department(did int primary key, dname varchar(30));
- create table employee(eid int primary key, ename varchar(30), dept_id int, foreign key(dept_id) references department(did));

MySQL Joins:

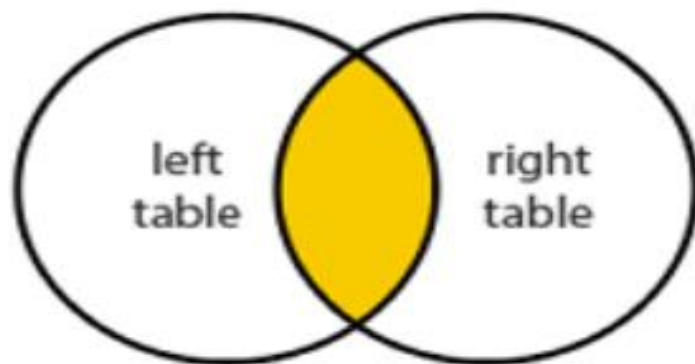
- A Join clause is used to combine rows from 2 or more tables , based on related column on them.
- Why joins are used?
- To fetch records from 2 or more tables .
- MySQL JOINS are used with SELECT statement.

Types of Joins:

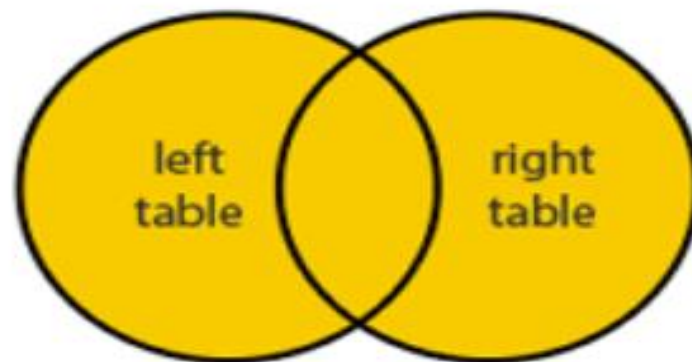


- INNER JOIN: Returns records that have matching values in both tables
- LEFT JOIN: Returns all records from the left table, and the matched records from the right table
- RIGHT JOIN: Returns all records from the right table, and the matched records from the left table
- Returns all records from both tables.
- SQL Full Join creates a new table by joining two tables as a whole.
- The joined table contains all records from both the tables and fills NULL values for missing matches on either side.
- Combination?

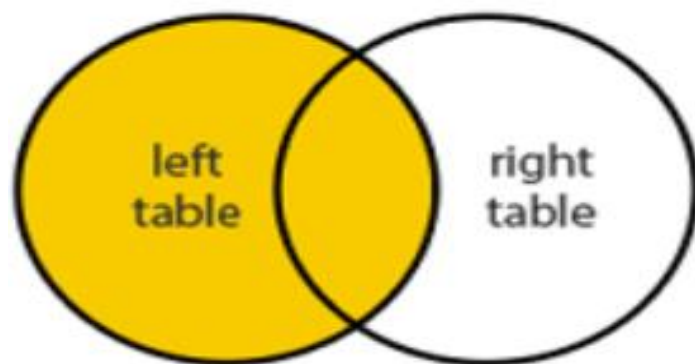
JOIN



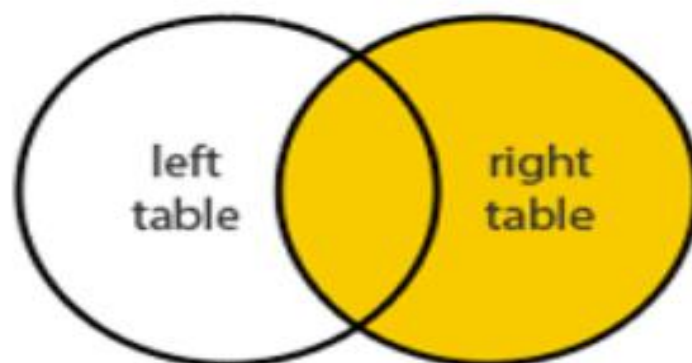
FULL JOIN



LEFT JOIN



RIGHT JOIN



Self Join: 1/05/24

- The self join, as its name implies, joins a table to itself.
- To use a self join, the table must contain a column that acts as the primary key and a different column within same table that have same datatype.
- Like 2 ids in same table.

Aliases

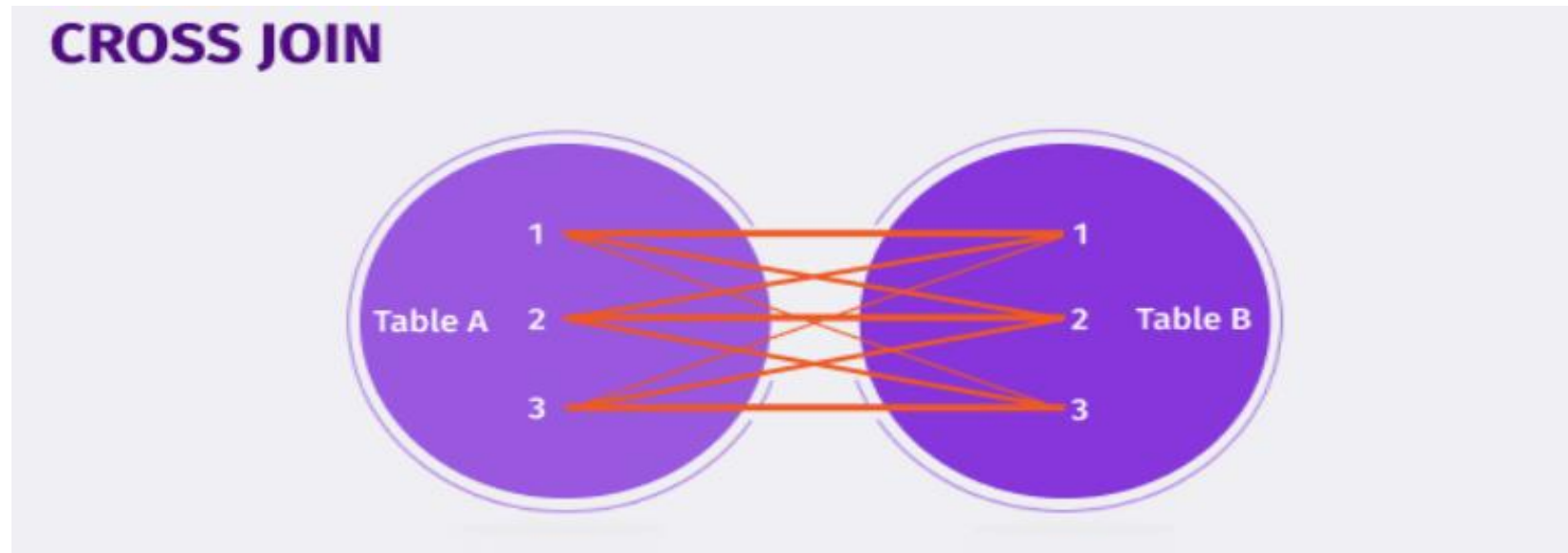
- Aliases are the temporary names given to table or columns in the table.
- To make table/column names more readable.
- An alias only exists for the duration of that query.
- 'as' keyword is used to create it. But it is optional.
- EX: ename as employee_name
- Employee as e.

- ```
SELECT C.ID, C.NAME, C.AGE, O.AMOUNT FROM
CUSTOMERS AS C, ORDERS AS O WHERE C.ID =
O.CUSTOMER_ID;
```



# Cross Join or Cartesian Join

- **CROSS JOINS** are used to combine each row of one table with each row of another table.
- The **CROSS JOIN** query in SQL is used to generate all combinations of records in two tables.



|   | Car_model |
|---|-----------|
| 1 | Camry     |
| 2 | Corolla   |
| 3 | Prius     |

|   | Color_name |
|---|------------|
| 1 | Black      |
| 2 | Red        |
| 3 | Silver     |

CROSS JOIN

|   | Car_model | Color_name |
|---|-----------|------------|
| 1 | Camry     | Black      |
| 2 | Corolla   | Black      |
| 3 | Prius     | Black      |
| 4 | Camry     | Red        |
| 5 | Corolla   | Red        |
| 6 | Prius     | Red        |
| 7 | Camry     | Silver     |
| 8 | Corolla   | Silver     |
| 9 | Prius     | Silver     |

# UNION operator: 2-05-24

- What Is UNION in SQL?
- It's an operator which is used to combine the data from the result of two or more SELECT command queries into a single distinct result set.
- It removes any duplicates present in the results being combined.
- Means?
- Syntax: Select column1, column2..... from table1

Union

Select column1, column2..... from table2

Constraint or rule: The number of fields & its datatype must be same for every SELECT statement connected with the UNION operator.

| Table 1  |          |
|----------|----------|
| Column 1 | Column 2 |
| A        | 3        |
| A        | 2        |
| A        | 1        |

**UNION**

| Table 2  |          |
|----------|----------|
| Column 1 | Column 2 |
| A        | 1        |
| B        | 2        |
| C        | 3        |

| Resulting Table |          |
|-----------------|----------|
| Column 1        | Column 2 |
| A               | 3        |
| A               | 2        |
| A               | 1        |
| B               | 2        |
| C               | 3        |

## Union All:

- UNION ALL Operator is used to combine result set of two or more SELECT queries.
- The UNION ALL operator does not remove duplicate rows from SELECT statement result set.
- Syntax: Select column1, column2..... from table1

Union All

Select column1, column2..... from table2

## Difference between Union & Union All

- UNION operator exclude duplicate rows from result set, where Union all retains it.

## Truncate Command:

- It is DDL command, that deletes the records in the table but it will not remove the table structure itself.
- Drop command is used to remove table structure and its contents.
- Delete command is used to delete a record or records from table with some condition mentioned in where clause.
- DROP TABLE *table\_name*;
- TRUNCATE TABLE *table\_name*;
- DELETE FROM *table\_name* WHERE condition;
- The truncate command works so fast that it does not have time to log its records in the transaction log.

# Differences between Delete & Truncate

## **Truncate**

- It removes all rows from a table.
- You can't rollback deleted records.
- Its faster than delete.
- In SQL, the auto increment counter get reset with truncate.

## **Delete**

- It removes a row or rows from a table along with some condition.
- You can rollback deleted records.
- The auto increment counter doesn't get reset with delete.



## Order By:03-05-24

- It is a keyword used to sort the records in ascending or descending order.
- Default sorting order is ascending order.
- To sort in descending order use **desc**;

- **Syntax:**

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;
```

## Distinct :

- To avoid duplicate values present in any specific columns/table, then **distinct** keyword is used.
- The distinct keyword is used in conjunction with the select keyword.
- Select DISTINCT column1, column2.....  
from table\_name.

# Aggregate functions:

- An aggregate function is a function that performs a calculation on a set of values, and returns a single value.
- It is used to summarize data.
- SQL Aggregate functions are mostly used with the GROUP BY clause of the SELECT statement.

The most commonly used SQL aggregate functions are:

- `MIN()` - returns the smallest value within the selected column
- `MAX()` - returns the largest value within the selected column
- `COUNT()` - returns the number of rows in a set
- `SUM()` - returns the total sum of a numerical column
- `AVG()` - returns the average value of a numerical column

Aggregate functions ignore null values (except for `COUNT()` ).

## Group By clause:

- Group By clause is used to arrange identical data (present into different rows) Into groups.
- Means: if a particular column has the same values in different rows then it will arrange these rows in a group.
- Syntax:  

```
select column names...
from table_name
group by column_name;
```
- It splits the result-set into groups of values & aggregate function can be used to return single value for each group.

# Features of Group By clause:

- GROUP BY clause is used with the SELECT statement.
- In the query, the GROUP BY clause is placed after the WHERE clause if used.
- In the query, the GROUP BY clause is placed before the ORDER BY clause if used.
- In the query, the Group BY clause is placed before the Having clause.
- Place condition in the Having clause.

## Limit: 04-05-24

- The LIMIT clause is used to set an upper limit on the number of records returned by query.
- It is used with select statement, the limit row count determine no. of rows returned by the query.
- When you use the LIMIT clause, it is important to use an ORDER BY clause to ensure the order of rows in the result set.

- Syntax:

```
select column names
from table name
order by column name limit l;
```

l is positive integer;

Limit:

|  | employee_id | first_name | last_name |
|--|-------------|------------|-----------|
|  | 121         | Adam       | Fripp     |
|  | 103         | Alexander  | Hunold    |
|  | 115         | Alexander  | Khoo      |
|  | 193         | Britney    | Everett   |
|  | 104         | Bruce      | Ernst     |
|  | 179         | Charles    | Johnson   |
|  | 109         | Daniel     | Faviet    |
|  | 105         | David      | Austin    |
|  | 114         | Den        | Raphaely  |



|   | employee_id | first_name | last_name |
|---|-------------|------------|-----------|
| ▶ | 121         | Adam       | Fripp     |
|   | 115         | Alexander  | Khoo      |
|   | 103         | Alexander  | Hunold    |
|   | 193         | Britney    | Everett   |
|   | 104         | Bruce      | Ernst     |

## OFFSET:

- It is a clause used to skip a certain number of rows when returning the result set of a query.
- It is used with the 'LIMIT' clause .
- Syntax:

select column names

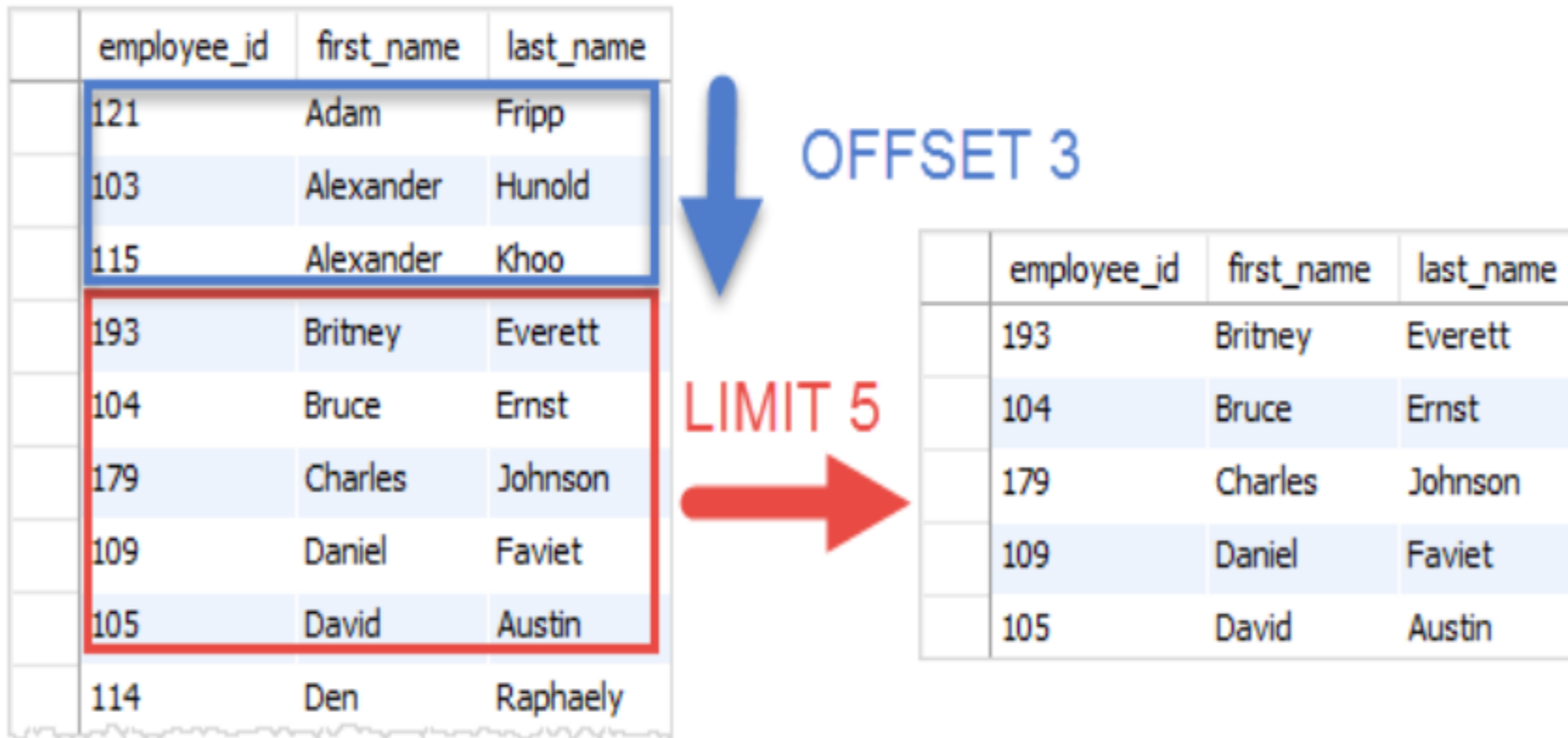
from table name

order by column name limit l, offset j;

l, j are positive integers.



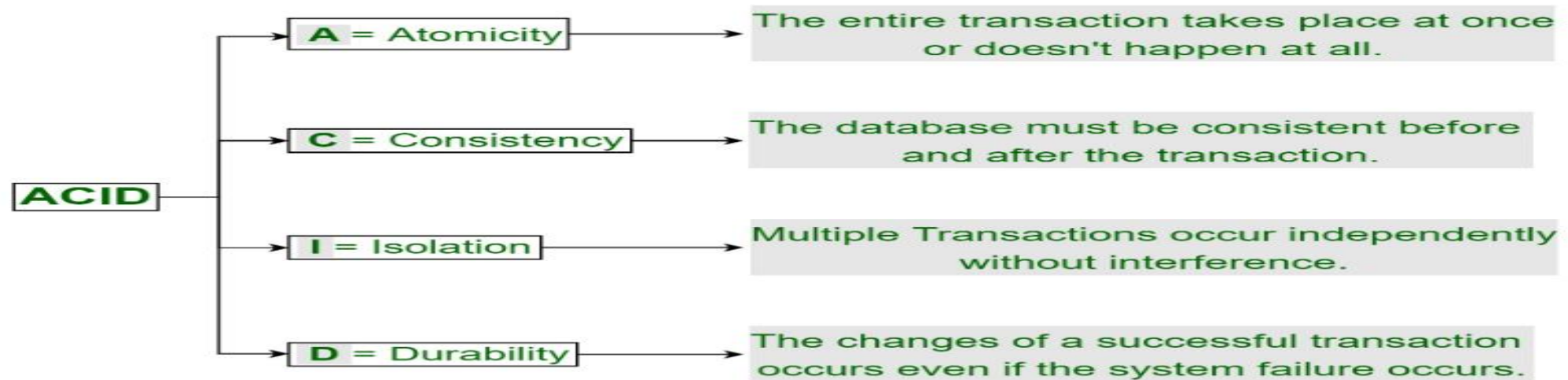
Offset:



# ACID Properties in DBMS:

- A **transaction** is a single logical unit of work that accesses and possibly modifies the contents of a database.
- In order to maintain consistency in a database, before and after the transaction, certain properties are followed. These are called **ACID** properties.

## ACID Properties in DBMS



## Default Constraint:

- It is used to specify default value for the column.
- If not any value specified to a particular field, then default value get added to all new records related to that particular column.

# Normalization:

- Normalization is the process of organizing data in a database.
- It includes creating tables and establishing relationships between those tables according to some rules.
- Advantages:
  1. To protect the data.
  2. To make the database more flexible
  3. Eliminating data redundancy
  4. Inconsistent dependency

# Redundancy & Inconsistent dependencies :

- **Redundancy** means having multiple copies of the same data in the database.
- Inconsistent dependencies can make data difficult to access because the path to find the data may be missing or broken.
- There are a few rules for database normalization. Each rule is called a "normal form."
- 1NF: each column of each table don't contain multiple values. Ex contact no.
- 2NF: It should follow 1<sup>st</sup> one & there should not be any partial dependencies. Ex : Employee id, dept. id(prime attribute), office location ( non-prime attribute).

## Normalization:

- 3NF: It should follow 2<sup>nd</sup>
- All non-prime attribute must depend on prime attributes.
- To make the data model more informative.
- Ex. Student id, subject id, subject in one table.

## 2NF:

| Employee Id | Department Id | Office Location |
|-------------|---------------|-----------------|
| 1EDU001     | ED-T1         | Pune            |
| 1EDU002     | ED-S2         | Bengaluru       |
| 1EDU003     | ED-M1         | Delhi           |
| 1EDU004     | ED-T3         | Mumbai          |

| Employee Id | Department Id |
|-------------|---------------|
| 1EDU001     | ED-T1         |
| 1EDU002     | ED-S2         |
| 1EDU003     | ED-M1         |
| 1EDU004     | ED-T3         |

| Department Id | Office Location |
|---------------|-----------------|
| ED-T1         | Pune            |
| ED-S2         | Bengaluru       |
| ED-M1         | Delhi           |
| ED-T3         | Mumbai          |

## 3NF:

- Student ID determines Subject ID, and Subject ID determines Subject. Therefore, Student ID determines Subject via Subject ID.
- This implies that we have a transitive functional dependency, and this structure does not satisfy the third normal form. Means non-key attributes should fully functional dependent only on the primary key.

| Student Id | Student Name | Subject Id | Subject | Address   |
|------------|--------------|------------|---------|-----------|
| 1DT15ENG01 | Alex         | 15CS11     | SQL     | Goa       |
| 1DT15ENG02 | Barry        | 15CS13     | JAVA    | Bengaluru |
| 1DT15ENG03 | Clair        | 15CS12     | C++     | Delhi     |
| 1DT15ENG04 | David        | 15CS13     | JAVA    | Kochi     |

| Student Id | Student Name | Subject Id | Address   |
|------------|--------------|------------|-----------|
| 1DT15ENG01 | Alex         | 15CS11     | Goa       |
| 1DT15ENG02 | Barry        | 15CS13     | Bengaluru |
| 1DT15ENG03 | Clair        | 15CS12     | Delhi     |
| 1DT15ENG04 | David        | 15CS13     | Kochi     |

| Subject Id | Subject |
|------------|---------|
| 15CS11     | SQL     |
| 15CS13     | JAVA    |
| 15CS12     | C++     |
| 15CS13     | JAVA    |



## Data control language:

- These commands are used to control user access to database objects and their contents.
- These commands mostly used by database security administrator that are admin user.
- Admin user must create & authorize other users who don't have authorities to see or do anything in DB.(System view)
- As admin user you grant users more privileges, they can access more database objects.
- With these privileges, they can retrieve lists of user database objects and select data within those objects.

## Grant:

- This command allows the administrator to provide particular privileges or permissions to the user over a database objects to perform certain operations on it.
- The primary function of the GRANT command in SQL is to provide administrators the ability to ensure the security and integrity of the data.
- `GRANT SELECT ON table_name TO user_name;`

## Revoke:

- Revoke means to take away.
- The REVOKE command enables the database administrator to remove the previously provided privileges or permissions from a user over a database or database object.
- `revoke SELECT ON table_name from user_name;`

## Advantages & Disadvantages of DCL:

| Advantages         | Disadvantages        |
|--------------------|----------------------|
| 1. Security        | Complexity           |
| 2. Control over DB | Time consuming       |
| 3. Flexibility     | Risk of human errors |

## Stored procedures:

- Stored procedures are prepared SQL code that you save so you can reuse it over and over again.
- If a query executes again & again then save it as a stored procedure and call it to run it.
- You can also pass parameters to stored procedures.
- Stored Procedures are created to perform one or more DML operations on Database.
- It is nothing but the group of SQL statements that accepts some input in the form of parameters and performs some task and may or may not return a value.

## Stored procedures:

- *CREATE PROCEDURE procedure\_name  
(parameter1 data\_type, parameter2 data\_type, ...)  
AS  
BEGIN  
    — SQL statements to be executed  
END*

# Delimiter:

- Delimiter helps to create the stored procedure. It is a special character that we can use to end a MySQL statement. This special character can be //, \$\$, @@, etc. It separates individual statements within a larger script, query, or stored procedure, allowing MySQL to interpret and execute them correctly.

## Function: 7-05-24

- These are user defined functions in SQL which accepts inputs, perform actions using those inputs, and return the output.
- It typically have at least four elements:
- A **name**, which is used to invoke the function
- **Arguments**, the inputs a function takes
- An output or **return** statement
- The **function body** itself



# Differences

## Stored Procedure

- It is precompiled.
- It doesn't have a return type, but it returns values using OUT parameter.
- It allows input & output parameters.
- It supports DML & DQL operations.
- Calling a function from stored procedure is possible.

## Functions

- Function needs to compile every time, when it get invoked.
- It has return type & it always return a value.
- It doesn't allow output parameters.
- It supports DQL operation only.
- You can't call stored procedure in function.

## Views: 7-05-24

- A VIEW in SQL is like a logical table that contains data from one or multiple tables.
- It does not hold any data and does not exist physically in the database. Its name should be unique in DB.
- It contains a set of predefined SQL queries to fetch data from the database.
- Advantage: we can control user security for accessing the data from the database tables. We can allow users to get the data from the VIEW, and the user does not require permission for each table or column to fetch data.

# Views Syntax

- CREATE VIEW Name AS

Select column1, Column2...Column N From tables

Where conditions;

- DROP VIEW view\_name;

## Index:

- An index in SQL Server is a data structure associated with a table or view that speeds up the retrieval of rows from the table based on the one or more columns.

- Syntax:

```
CREATE INDEX name_of_Index ON name_Of_Table(Attribute1,
Attribute2,...);
```

## Trigger:

- A trigger is a stored procedure in a database that automatically invokes whenever a special event in the database occurs.
- a trigger can be invoked when a row is inserted into a specified table or when specific table columns are updated.
- a trigger is a collection of SQL statements with particular names that are stored in system memory.
- A trigger is automatically called whenever a data modification event against a table takes place, which is the main distinction between a trigger and a procedure.

# Trigger:

- *create trigger [trigger\_name]*  
*[before | after]*  
*{insert | update | delete}*  
*on [table\_name]*  
*[for each row]*  
*[trigger\_body]*